SCALER School of Technology | Priyansh Saxena

# MLOps: Experiment Tracking, Model Registry, and CI/CD for ML

Reproducibility ▪ Registry Lifecycle ▪ CI/CD Gates ▪ Distribution Shift

**Priyansh Saxena**

Scaler School of Technology | ML System Design and LLMOps

Week 3 — Session 2/2

ROADMAP

# Agenda

1. Experiment Tracking
2. Model Registry
3. CI/CD for ML
4. Distribution Shift and Monitoring

TOC

# 1

# Experiment Tracking



# 1 The Problem: ML Without a Lab Notebook

**Sound familiar?**

- "Did we try dropout = 0.3? What was the result?"
- "Which dataset version did the best model train on?"
- "Why did experiment 47 beat experiment 46?"
- "Can you reproduce last month's model exactly?"

## The cost of not tracking

You cannot reproduce your best result. You cannot explain what changed. You cannot hand your work to a teammate. You are starting from scratch every time.

## What to track per experiment

- **Loss curves** — train, val, test
- **All metrics** — accuracy, F1, AUC, NDCG
- **Hyperparameters** — every knob you turned
- **Data version** — DVC tag or dataset hash
- **Code version** — git commit hash
- **Artefacts** — model file, preprocessor, config
- **System metrics** — GPU memory, training time
- **Sample predictions** — for sanity checks

# 1 MLflow: Your ML Lab Notebook

**Core concepts**

- **Experiment**: a named collection of runs (e.g. "fraud-detection-v2")
- **Run**: one execution of your training script
- **Parameters**: inputs — hyperparameters, config
- **Metrics**: outputs — logged over time (loss per epoch)
- **Artefacts**: files — model, plots, confusion matrices
- **Tags**: free-form metadata (git hash, author, dataset name)

## Minimal MLflow tracking

```
import mlflow
mlflow.set_experiment("fraud-v2")

with mlflow.start_run():
  mlflow.log_params(config)
  for epoch in range(N):
    mlflow.log_metric(
      "val_auc", auc, step=epoch)
  mlflow.sklearn.log_model(clf,
"model")
  mlflow.set_tag(
    "git_commit", get_git_hash())
```

*Tools: MLflow (open source), Weights & Biases (hosted), Neptune, Comet*

# 1 Reproducibility: Why It's Hard and How to Fix It

**Sources of nondeterminism**

- **GPU parallel ops** — CUDA operations are not deterministic by default; set `torch.use_deterministic_algorithms(True)`
- **Floating-point rounding** — differs across hardware and library versions
- **Random seeds** — data shuffling, weight initialisation, dropout
- **Library version drift** — PyTorch 2.0 vs. 2.1 may give different results
- **Data mutation** — if training data changes after the run, you can't reproduce

## Reproducibility checklist

- ✓ Set all random seeds (`torch`, `numpy`, `random`, `os.environ["PYTHONHASHSEED"]`)
- ✓ Pin library versions (`requirements.txt` or `conda env`)
- ✓ Version data with DVC and log the tag
- ✓ Log git commit hash with every run
- ✓ Use Docker for environment reproducibility

*Tools: DVC (data versioning), Docker (environment), MLflow (experiment context)*

2

# Model Registry

# 2 Registry vs. S3: A Tale of Two Artefact Stores

## The S3 approach

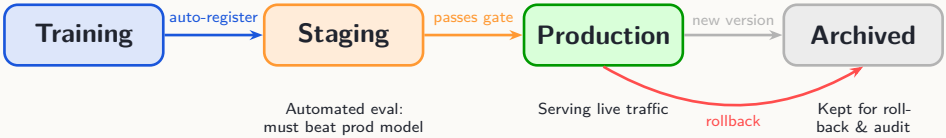
```
fraud_model_v2.pkl
fraud_model_v3.pkl
fraud_model_v3_final.pkl
fraud_model_v3_final_FINAL.pkl
fraud_model_PROD_USE_THIS.pkl
```

No versioning. No lineage. No lifecycle. No rollback. No audit trail. When something breaks in production: "Which one is live? Who deployed it? When?"

## The registry approach

| Property | Value |
|---|---|
| Version | v3.2.1 |
| Stage | Production |
| Trained on | dataset-2024-11 |
| Git commit | a3f91bc |
| Val AUC | 0.9847 |
| Promoted by | psaxena |
| Promoted at | 2024-11-15 14:32 |
| Previous | v3.1.0 (Archived) |

# 2 Model Registry Lifecycle




## Promotion rules (example)

- New model must beat production NDCG@20 by $\geq 1\%$
- No regression on critical slices (high-value users)
- Latency p99 must stay $\leq$ 100ms
- Diversity score must not drop $> 5\%$

## Rollback in one command

```
# Demote bad model
client.transition_model_version_stage(
  "fraud-detector", "3", "Archived")

# Promote previous
client.transition_model_version_stage(
  "fraud-detector", "2", "Production")
```

# 5-Minute Break

Time to step away from the experiment tracker.
Grab some water, stretch, and come back refreshed.

Your models can train without you. See you in five.

3

# CI/CD for ML

# 3 CI/CD for ML: Beyond Code Testing

**Standard software CI/CD**

- Unit tests — does the code work?
- Integration tests — do components talk?
- Build — does it compile/package?
- Deploy to staging, then production

**ML adds three new concerns**

1. **Data validation** — is the incoming data still valid?
2. **Model evaluation** — is the new model actually better?
3. **Training regression** — does training still converge correctly?

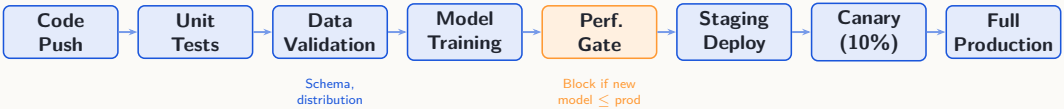
## ML-specific test examples

**Data validation (Great Expectations):**

- Schema: expected columns, dtypes
- Null rate per column $\leq$ threshold
- Value range checks (age $\in [0, 130]$)
- Volume: sudden drop = pipeline failure
- Distribution: KL divergence vs. baseline

**Model performance gate:**
New model must beat production on held-out eval set before any deployment proceeds.

# 3 The ML CI/CD Pipeline




## What triggers the pipeline?

- Code push to main branch
- New data batch available (scheduled)
- Drift alert from monitoring
- Manual trigger (on-demand retraining)

## Tools

- Orchestration: Airflow, Kubeflow Pipelines, Prefect
- Data validation: Great Expectations, dbt tests
- CI trigger: GitHub Actions, Jenkins
- Environment: Docker + Kubernetes

# 3 Quiz 1

## Q1 — The Promotion Gate

You are building a CI/CD pipeline for a fraud detection model. The current production model has a val AUC of 0.9847 on a held-out test set. A new model achieves val AUC of 0.9851 on the same test set.

**Should the new model be automatically promoted? What additional checks would you require before allowing promotion?**

*Hint: think about sliced performance, latency, and the risk of overfitting to your eval set.*

## Q2 — The Broken Pipeline

You push a code change that refactors feature computation. The CI pipeline passes unit tests but the model performance gate blocks: val AUC dropped from 0.9847 to 0.8312.

**List your debugging steps in order. What do you check first?**

# 3 Quiz 1 — Answers

## A1 — The Promotion Gate

**Not automatically — 0.0004 AUC improvement is within noise.** Additional checks required before promotion:

**(1) Sliced performance:** does the new model maintain performance on high-value users, rare fraud patterns, and underrepresented demographic groups? A model can improve on average while regressing on a critical slice.

**(2) Latency:** does p99 inference latency stay $\leq$ 100ms? A more complex model might be marginally better but breach SLA.

**(3) Eval set freshness:** if the held-out test set is months old, a small improvement may reflect overfitting to the old distribution rather than genuine improvement.

## A2 — The Broken Pipeline

**Check in this order:** (1) **Feature computation diff** — did the refactor change any feature value, even for one column? Print a sample of 10 rows from the old vs. new pipeline and compare. (2) **Data leakage direction** — did you accidentally *remove* leakage (i.e. the old pipeline was leaking future data, inflating AUC)? (3) **Scaling stats** — are you still fitting the scaler on training data only, or did the refactor fit on all data? (4) **Git diff** of the feature code to identify the exact change.

# 4

# Distribution Shift and Monitoring

# 4 The Three Types of Distribution Shift

| Type | Definition | Real-world example |
|---|---|---|
| **Covariate shift** | $P(X)$ changes; $P(Y\|X)$ stays the same | Credit scoring model trained pre-COVID; post-COVID income distributions are completely different |
| **Label shift** | $P(Y)$ changes; $P(X\|Y)$ stays the same | Spam filter: spammer tactics evolve; the spam category contains different emails each year |
| **Concept drift** | $P(Y\|X)$ changes; same input, different correct output | "Wuhan" search queries in 2019 = travel information; in 2020 = COVID origin. Same input, completely different user intent. |

## The uncomfortable truth about drift alerts

A monitoring company CTO: "In my estimate, 80% of the drifts captured by our service are caused by human errors" — bugs in data pipelines, missing values incorrectly inputted, wrong feature statistics used for scaling, training-serving skew. Fix your pipeline before blaming the world.

# 4 Monitoring: Four Layers

**Layer 1 — Data quality** (check first)

- Schema violations (unexpected column, wrong dtype)
- Null rate spikes per feature
- Volume drops (pipeline failure signal)
- Value range violations

**Layer 2 — Feature drift**

- Input distribution vs. training baseline
- Statistical tests: KS test, PSI (population stability index), Jensen-Shannon divergence
- Tools: Evidently AI, WhyLabs, Arize

**Layer 3 — Prediction drift**

- Output distribution shift
- For classification: predicted class distribution
- For regression: predicted value distribution
- Alert: if distribution diverges significantly from deployment baseline

**Layer 4 — Business metrics**

- CTR, conversion rate, revenue
- These are slow to respond but carry the most signal
- Alert: 5% drop week-over-week in primary KPI

## Alert ordering matters

Data quality alert → fix pipeline. Feature drift alert → consider retraining. Business metric drop → urgent investigation. Don't wait for Layer 4.

# 4 Retraining Triggers

**1. Scheduled retraining**

- Retrain weekly/monthly regardless of detected drift
- Simple to implement and reason about
- Risk: too infrequent for fast-moving domains (news, finance); too frequent = wasted compute for stable domains (medical imaging)

**2. Drift-triggered retraining**

- Statistical test fires when $P(X)$ diverges beyond threshold
- More adaptive but can fire on false alarms (80% are pipeline bugs!)
- Must have a pipeline bug checklist that runs first

**3. Performance-triggered retraining**

- Business metric drops below threshold (e.g. CTR drops $> 5\%$ week-over-week)
- Most trustworthy signal but slowest to detect
- Requires ground truth labels — not always available immediately

## Choosing your strategy

| Domain | Strategy |
|---|---|
| News rec. | Drift-triggered |
| Fraud detection | Performance-triggered |
| Medical imaging | Scheduled (quarterly) |
| Credit scoring | Scheduled + regulatory |

# 4 Quiz 2

## Q3 — Diagnosing a Production Drop

A recommendation system's CTR has dropped 8% over the past two weeks. Latency and error rates are unchanged. The monitoring dashboard shows feature drift alerts on three user features: `last_session_duration`, `days_since_last_login`, and `purchase_count_30d`.

**Walk through your debugging process step by step. What do you check first? What would confirm this is a data pipeline bug vs. genuine drift? When do you decide to retrain?**

## Q4 — Choosing a Retraining Strategy

You are building the monitoring strategy for (a) a real-time fraud detection model and (b) a quarterly employee performance review model.

**For each system: what type of drift is most likely? Which retraining trigger would you use? What is the main risk of your chosen approach?**

# 4 Quiz 2 — Answers

## A3 — Diagnosing a Production Drop

**Step 1: Check for pipeline bugs first** (80% of drift alerts are bugs). Were the three features recently changed in the ETL? Check for: null rates (sudden spike = missing upstream data), value range changes (e.g. `purchase_count_30d` now returning 0 for everyone = query bug), timestamp errors (features computed with wrong date window).

**Step 2: Confirm genuine drift** if no pipeline bug. Compare feature distributions week-over-week using PSI or KS test.

**Retrain when:** pipeline is healthy, drift is confirmed statistically, and a retrain on recent data shows improved offline metrics on a fresh held-out set.

## A4 — Choosing a Retraining Strategy

**Fraud detection:** most likely concept drift (fraudsters adapt tactics). Use **performance-triggered** retraining — monitor precision/recall on flagged transactions daily; retrain when recall drops below threshold. Main risk: ground truth labels are delayed (chargebacks take days), so you may retrain on incomplete labels.

**Employee performance review:** most likely covariate shift (workforce demographics change slowly). Use **scheduled** retraining quarterly, aligned with review cycles. Main risk: too infrequent if the company undergoes rapid restructuring or hiring.

# 5 What We Covered Today

## §1 Experiment Tracking

- Track everything: params, metrics, data version, code version, artefacts
- MLflow and W&B as your lab notebook
- Reproducibility checklist: seeds, versions, Docker

## §2 Model Registry

- Registry $\neq$ S3 bucket
- Lifecycle: Staging $\rightarrow$ Production $\rightarrow$ Archived
- Full lineage, audit trail, one-command rollback

## §3 CI/CD for ML

- ML adds: data validation + model perf gates
- New model only promoted if it beats production
- Full pipeline: code push $\rightarrow$ production

## §4 Distribution Shift and Monitoring

- 3 shift types: covariate, label, concept
- 80% of apparent drift = pipeline bugs
- 4-layer monitoring: data $\rightarrow$ features $\rightarrow$ predictions $\rightarrow$ business
- 3 retraining triggers: scheduled, drift, performance

# Thank You!

An untracked experiment is a forgotten experiment.
An unmonitored model is a ticking clock.

**Priyansh Saxena**

All materials at:

**https://www.priyanshsaxena.com/sst-ml-system-design**

